



Nexus Tools

Manual

typst

nexus-tools

0.2.0

MIT

Maycon F. Melo*

Contents

Description	2
Main	2
Custom Defaults	3
Content to String	3
Storage	3
Set Namespace	4
Add Data	4
Remove Data	4
Get Data	5
Get Final Data	5
Reset Database	6
Components	6
Paper-friendly URLs	6
Package URLs	6
Callout	7
Get	7
Null Value	8
Automatic Values	8
Date	8
Relative Luminance	9

*<https://github.com/mayconfmelo>

Dynamic colors	9
Has	9
Field	10
Key	10
Value	10
Item	10
It's	11
None	11
Null	11
Empty	11
Context	11
Sequence of Contents	12
Space Content	12
Function	12
Type	12
Copyright	13

Description

Easily implement handy features from a curated collection of frequently used package functionality, such as data storage or custom defaults. This library was created as part of the development of *my other Typst projects*;¹ it contains functionality shared across multiple projects that would otherwise need to be maintained and updated individually, but is now centralized in a single place. This is not intended to be a full-fledged development toolset, but rather a compartmentalization of shared resources.

You can use this library without restrictions as a development aid, especially for packages.

Main

```
#import "@preview/nexus-tools:0.2.0"
```

¹<https://typst.app/universe/search/?q=author%3A%22Maycon%20F.%20Melo%22>

Custom Defaults

```
#default(
  when: false,
  value: (:),
  otherwise: (:),
  disabled,
)
```

Substitutes Typst defaults for custom ones, allowing to set new defaults that can be easily changed using `#set` rules; it works by mapping custom defaults to the originals, replacing them when in use.

when: `boolean`

Condition to apply custom default.

value: `dictionary` `any`

Custom default value (condition is `true`).

otherwise: `dictionary` `any`

Alternative value when default is not being used (condition is `false`).

disabled `boolean`

Disable custom default when `true`.

This command is commonly used inside `#set` rules.

Content to String

```
#content2str(data)
```

Converts content to string.

data `content`

Content data

Storage

```
#import "preview/nexus-tools:0.2.0": storage
```

Set Namespace

```
#storage.namespace(name)
```

Set the namespace used as storage. Namespaces allows multiple packages/templates to use `#storage` at the same time, each accessing its own proper space. This changes where data is stored **globally**, use it with caution.

name string	<i>(required)</i>
Namespace name.	

Add Data

```
#storage.add(
    key,
    value,
    append: false,
    namespace: auto,
)
```

Insert a new entry in the storage.

key string
Storage entry name.

value ant
Value to be stored.

append: boolean
If entry already exists, append value when true; replaces it when false.

namespace: string
Add to the given namespace.

Remove Data

```
#storage.remove(
    key,
    namespace: auto,
)
```

Removes an existing entry from storage.

key string

Storage entry name.

namespace: string

Remove from the given namespace.

Get Data

```
#storage.get(
    ..args,
    namespace: auto,
)
```

Retrieves a value from storage, or the entire namespace itself.

args.pos().at(0) string

Storage entry name; if not set, returns the whole namespace.

args.pos().at(1) any

Default value returned when the storage entry is not found; otherwise returns none.

namespace: string

Get from the given namespace.

Get Final Data

```
#storage.final(
    ..args,
    namespace: auto,
)
```

Retrieves the final value (or namespace) state from storage.

args.pos().at(0) string

Storage entry name; if not set, returns the final state for the whole namespace.

args.pos().at(1) any

Default value returned when the storage entry is not found; otherwise returns none.

namespace: string

Final state from the given namespace.

Reset Database

```
#storage.reset(
    data,
    namespace: auto,
)
```

Set a new value for the entire storage namespace. While the new value can be of any type, the other storage commands can only work with dictionary values.

data dictionary any
New namespace value.

namespace: string
Reset the given namespace.

Components

```
#import "@preview/nexus-tools: 0.1.0": comp
```

Paper-friendly URLs

```
#comp.url(target, id, text)
```

Creates a paper-friendly link, attached to a footnote containing the URL itself to ensure readability when printed.

target string label *(required)*
URL used as link and footnote, or label referencing a previous `#url` command.

id label
Optional label for further reference.

text string content
Text shown as link (fallback to the URL).

Package URLs

```
#comp.pkg(target, id)
```

Generates paper-friendly links for general package/library repositories.

target string label`"https://example.com/name"``"https://example.com/{name}/slug/"`

Package URL, or label referencing a previous `#pkg` command. The package name is inferred from the last URL slug or retrieved from curly brackets.

id label

Optional label for further reference.

Callout

```
#comp.callout(
  icon: "information-circle",
  title: none,
  text: (:),
  background: (:),
  body,
)
```

Creates a highly customizable callout box.

icon: string

Icon name, as set by *Heroicons*².

title: string content none

Title, if any.

text: color dictionary

`#text` options (the special `#text.title` is used for title).

background: color dictionary

`#block` options.

Get

```
#import "@preview/nexus-tools: 0.1.0": get
```

²<https://heroicons.com/>

Null Value

```
#get.null
```

A null value that matches only itself. Useful as substitute for none default values, as it is truly unique and not naturally returned by anything in Typst.

Automatic Values

```
#get.auto-val(
  origin,
  replace,
)
```

Sets a meaningful value to auto values: if a value is automatic, returns the replacement; otherwise, returns the value unchanged.

origin `auto` `any`

Value to be checked.

replace `any`

Replacement for the value if auto.

Date

```
#get.date(..date, pattern: "mm/dd/yyyy")
```

Create a `#datetime` date from named and/or positional arguments, combined or not. Panics if two values are set for the same data component — like a positional and also a named value for the year.

..date `arguments` `array` `dictionary` `string` *(required)*

(year, month, day)

Components of the data, as positional/named arguments, array, dictionary, or string; fallback to current year, month 1, and day 1 when these components are not set.

pattern: `string`

Pattern used to parse datetime from string.

Relative Luminance

```
#relative-luminance(color)
```

Calculates the relative relative luminance of a color (0 is lighter and 1 is darker)

color `color`

Color to be evaluated.

#relative-luminance() → `float`

The relative luminance is expressed as a floating-point number (0 is lighter and 1 is darker).

Dynamic colors

```
#get.dynamic-color(
  background,
  dark: white,
  light: black,
  limit: 0.55
)
```

Return a foreground color based on a background color luminance: lighter color when dark and darker color when light.

background `color`

Background color evaluated.

dark: `color`

Foreground color when background is darker.

light: `color`

Foreground color when background is lighter.

limit: `float`

Luminance limit (0 is light and 1 is dark).

Has

```
#import "@preview/nexus-tools: 0.1.0": has
```

Field

```
#has.field(data, values)
```

Check if content has a given field.

data content

The content itself.

values string array of strings

One or more field names.

Key

```
#has.key(data, values)
```

Check if dictionary or module has a given key.

data dictionary module

The dictionary or module itself.

values string array of strings

One or more key names.

Value

```
#has.value(data, values)
```

Check if dictionary or module has a given value.

data dictionary module

The dictionary or module itself.

values string array

One or more values.

Item

```
#has.item(data, values)
```

Check if an array has one or more given items.

data string

The array itself.

values string array of strings

One or more items.

It's

```
#import "@preview/nexus-tools: 0.1.0": its
```

None

```
#its.none-val(data)
```

Check whether a value is none.

data none any

Value to be checked.

Null

```
#its.null()
```

Check whether a value is #get.null.

data any

Value to be checked.

Empty

```
#its.empty(data)
```

Check whether a value is empty: "" or [] or () or (:).

data any

Value to be checked.

Context

```
#its.context-val(data)
```

Check whether a value is a `#context()`.

data any

Value to be checked.

Sequence of Contents

`#its.sequence(data)`

Check whether a value is a sequence of contents.

data any

Value to be checked.

Space Content

`#its.space(data)`

Check whether a value is a content with just a space.

data any

Value to be checked.

Function

`#its.func(data, values)`

Check whether a value function is one of the given ones.

data any

Value to be checked.

values function array of functions

One or more functions.

Type

`#its.type(data, values)`

Check whether a value type is one of the given ones.

data any

Value to be checked.

values type array of types

One or more types.

Copyright

Copyright © 2026 Maycon F. Melo.

This manual is licensed under MIT.

The manual source code is free software: you are free to change and redistribute it.

There is NO WARRANTY, to the extent permitted by law.

The logo was obtained from Flaticon website.

The Fluent support is a fork of *alinguify*³ feature, and all the overall project concept is heavily inspired in this great package.

³<https://typst.app/universe/package/linguify>